

Minősegbiztosítási Kérdőív

Tartalomjegyzék

1. Rendszerarchitektúra és Technológiák	2
2. Adatbázis Részletes Leírása	2
3. Hitelesítés és Jogosultságkezelés.....	3
4. API Végpontok Részletes Specifikációja	3
4.1. Hitelesítési modul.....	3
4.2. Diák Funkciók.....	4
4.3. Adminisztrátori Funkciók.....	4
4.3.1. Rendszerstátusz és Statisztika	4
4.3.2. Karbantartási funkciók (CRUD)	5
4.3.3. Segéd-erőforrások	5
5. Hibakezelés és Tesztelhetőség.....	6
1. Technológiai Áttekintés	7
2. Alkalmazásszerkezet és Útvonalvédelem (App.js)	7
3. Komponensek és Oldalak Elemzése	8
3.1. Bejelentkezés Kezelése (LoginPage, AdminLoginPage)	8
3.2. Értékelési Folyamat (Diák oldal)	8
3.3. Adminisztrációs Modulok	8
4. UI/UX és Közös Megoldások.....	9

Minősbiztosítási Kérdőív – Backend Fejlesztői Dokumentáció

1. Rendszerarchitektúra és Technológiák

A Minősbiztosítási Kérdőív backend rétege a **Laravel** keretrendszerre épül, amely egy modern, PHP-alapú MVC (*Model-View-Controller*) architektúrát biztosít.

- **RESTful API:** A rendszer szabványos HTTP metódusokon keresztül kommunikál a frontenddel (*React*).
- **Laravel Sanctum:** Token alapú hitelesítés, amely lehetővé teszi a biztonságos állapotatlan (*stateless*) kommunikációt.
- **Middleware-ek:** Egyedi szűrők gondoskodnak a jogosultságok ellenőrzéséről és a kérdőív állapotának (nyitva/zárva) felügyeletéről.
- **Adatbázis:** MySQL relációs adatbázis, amely biztosítja az adatok integritását és az anonimitást.

2. Adatbázis Részletes Leírása

Az adatbázis felépítése során az elsődleges szempont az adatok szétválasztása volt, hogy a statisztikák ne legyenek közvetlenül összeköthetők a kitöltő személyével.

Főbb táblák és struktúra

diak: A diákok nevét és OM azonosítóját tárolja. Az utóbbi az *id*, amely egyedi indexszel rendelkezik a gyors keresés érdekében.

csoport: A diákok és tanárok csoportja. Mezők: *id, nev, created_at, updated_at*.

tanar_csoport, diak_csoport: Kapcsolótáblák a *tanar* és *csoport* valamint a *diak* és a *csoport* táblák között.

admin: Az adminisztrátori hitelesítésre szolgáló tábla (felhasználónév és titkosított jelszó).

tanar: Az oktatók listája. Mezők: *id, nev, created_at, updated_at*.

kerdesek: A kérdőív vázát adó kérdések. Minden kérdéshez tartozik egy *tipus_id*, amely meghatározza a válaszlehetőségek skáláját.

ertekeles: Ez a központi kapcsolótábla. Itt rögzül, hogy melyik diák melyik tanárt értékelte. Tartalmaz egy unique(*diak_id, tanar_id*) kényszert, ami megakadályozza a duplikált kitöltést.

valaszok: A kitöltött kérdőívek adatait tárolja. A timestamp mezők NULL értékre vannak állítva az anonimitás érdekében. Mezők: *id, tanar_id, kerdes_id, ertek (0-4) created_at (NULL), updated_at (NULL)*.

3. Hitelesítés és Jogosultságkezelés

A rendszer két különböző jogosultsági szintet (képességet) kezel a Sanctum tokeneken keresztül.

- **role-diak:** Csak az őt tanító tanárok listájához és az értékelés beküldéséhez fér hozzá.
- **role-admin:** Hozzáférést kap a statisztikákhoz, a kérdések szerkesztéséhez és a rendszer állapotának vezérléséhez.

4. API Végpontok Részletes Specifikációja

4.1. Hitelesítési modul

Minden végpont az /api prefix alatt érhető el. Sikeres hitelesítés után a szerver egy *Plain Text Token*-t küld vissza (*Laravel Sanctum*), amelyet a kliensnek minden további kérés fejlécében (*Authorization: Bearer {token}*) szerepeltetnie kell.

Metódus	Végpont	Leírás	Bemeneti paraméterek	Válasz (Siker)
POST	/diak/login	Diák bejelentkezés OM azonosítóval.	diak_id (int)	200 OK + token & diák adatok
POST	/admin/login	Adminisztrátori belépés (felhasználónév/jelszó).	username, password	200 OK + token
POST	/diak/logout	Aktuális munkamenet (token) lezárása.	-	200 OK

Működés: Sikeres hitelesítés után a szerver egy **Plain Text Token**-t küld vissza, amelyet a kliensnek minden további kérés fejlécében (*Authorization: Bearer {token}*) szerepeltetnie kell.

Megjegyzés: A diák bejelentkezés csak akkor engedélyezett, ha a kérdőív állapota aktív a gyorsítótárban (*kerdoiv_aktiv*). Ellenkező esetben a rendszer 403 Forbidden hibát ad vissza.

4.2. Diák Funkciók

- **GET /api/hianyzo-ertekelesek:**
 - **Feladat:** Azon tanárok listázása, akiket a bejelentkezett diák még nem értékelt, de tanítja őt.
 - **Logika:** A lekérdezés összekapcsolja a *tanar* és *tanar_csoport* táblákat, szűrve a diák saját csoportjaira, és kizárva (*WHERE NOT IN*) azokat a tanárokat, akikhez már tartozik bejegyzés az ertekeles táblában a diák azonosítójával.
 - **Válasz:** Tanárok listája (*id, nev*).
- **POST /api/tanar/{id}/kerdesek:**
 - **Feladat:** Az értékeléshez szükséges kérdések és a válaszlehetőségek lekérése.
 - **Logika:** Csak a "diák" típusú kérdéseket adja vissza.

4.3. Adminisztrátori Funkciók

Ezek a végpontok kizárólag role-admin képességgel rendelkező tokennel érhetőek el.

4.3.1. Rendszerstátusz és Statisztika

- **GET /api/admin/kerdoiv-statusz**
 - **Feladat:** A kérdőív aktuális globális állapotának lekérdezése.
 - **Működés:** A *Cache::get('kerdoiv_aktiv')* értékét adja vissza. Ha nincs beállítva, alapértelmezetten false.
- **POST /api/admin/kerdoiv-statusz**
 - **Feladat:** A kérdőív kitöltésének engedélyezése vagy tiltása.
 - **Bemenet:** *{"aktiv": true/false}*
 - **Megvalósítás:** A *Cache::forever* metódussal tárolja az állapotot. Ez az érték határozza meg, hogy a diákok be tudnak-e jelentkezni az értékelő felületre.
- **GET /api/admin/stat/tanar/{tanarID}**
 - **Feladat:** Egy adott tanárra vonatkozó részletes statisztika generálása kérdésenkénti bontásban.
 - **Logika:** Összekapcsolja a *valaszok* és *kerdesek* táblákat.
 - **Szűrés:** Csak az adott *tanar_id*-hoz tartozó és 0-nál nagyobb (érvényes) pontszámokat veszi figyelembe.
 - **Kalkuláció:** *ROUND(AVG(ertek), 2)* és *COUNT()* függvényekkel kérdésenként csoportosítva.
 - **Hibaüzenet:** Ha a megadott azonosítóval nem létezik tanár, *404 Not Found* választ ad.

4.3.2. Karbantartási funkciók (CRUD)

Metódus	Végpont	Kontroller	Leírás
GET	/api/tanarok	TanarController	Az összes tanár listázása statisztikai választáshoz.
GET	/api/kerdesek	KerdesController	Az adatbázisban szereplő összes kérdés lekérése.
POST	/api/kerdesek	KerdesController	Új kérdés rögzítése (szöveg és típus megadása).
PUT	/api/kerdesek/{id}	KerdesController	Meglévő kérdés szövegének vagy paramétereinek módosítása.
DELETE	/api/kerdesek/{id}	KerdesController	Kérdés végleges törlése.

Adatintegritási szabály a törléshez:

A *KerdesController@destroy* metódus implementálásakor kötelező ellenőrizni, hogy a törölni kívánt kérdéshez tartozik-e már leadott válasz a válaszok táblában:

1. Ha létezik kapcsolódó válasz: A törlés megtagadva (*422 Unprocessable Entity*), mivel a törlés megbontaná a korábbi statisztikákat.
2. Ha nem létezik válasz: A rekord törölhető.

4.3.3. Segéd-erőforrások

- **GET /api/kerdes-tipusok:** Visszaadja a kérdésekhez rendelhető kategóriákat (pl. diák, szülői, egyéb), segítve az új kérdések létrehozását a frontenden.
-

5. Hibakezelés és Tesztelhetőség

200 OK / 201 Created

Sikeres válasz esetén a rendszer minden esetben JSON formátumban válaszol, tartalmazva a message kulcsot a frontend visszajelzésekhez.

422 Unprocessable Content (Validációs hiba)

A Laravel automatikus validátora ellenőrzi a bemeneteket.

- Példa: Ha a pontszám nem 0 és 4 közötti szám, a válasz tartalmazza a konkrét mező hibáját.

401/403 (Jogosultsági hiba)

- **401:** A token érvénytelen vagy hiányzik.
- **403:** A diák próbál admin végpontot hívni, vagy a kérdőív állapota "Zárva".

Fejlesztői Megjegyzések és Biztonság

Biztonsági protokoll: Az admin jelszavak *Bcrypt* algoritmussal kerülnek titkosításra. A rendszer védett az *SQL Injection* ellen az *Eloquent Query Builder* automatikus paraméter-kötései (*binding*) révén. A statisztikai adatok aggregáltak, így a tanárok nem láthatják, melyik diák adott konkrét pontszámot, biztosítva az értékelés valódi anonimitását.



Minőisegbiztosítási Kérdőív - Frontend Fejlesztői Dokumentáció

1. Technológiai Áttekintés

A kliensoldali alkalmazás egy **React-alapú Single Page Application (SPA)**, amely a következő főbb megoldásokat alkalmazza:

- **Navigáció:** *react-router-dom* (v6+) dinamikus útvonalválasztással és beágyazott útvonalakkal (*Outlet*).
- **Stíluskezelés:** *Bootstrap* osztályok a reszponzivitásért, valamint egyedi *.css* modulok a finomhangolásért.
- **Ikonkészlet:** *react-bootstrap-icons* a vizuális tájékozódás segítésére.

2. Alkalmazásszerkezet és Útvonalvédelem (App.js)

Az alkalmazás központi vezérlője az *App.js*, amely meghatározza a felhasználói szinteket és a hozzáférési szabályokat.

PrivateRoute Komponens

Útvonal-hierarchia

- **Publikus:** */login*, */admin/login*.
- **Védett Diák:** */hianyzo-ertekelesek*, */tanar/:tanarId/ertekeles*.
- **Védett Admin:** */admin/dashboard* és annak alútvonalai (*kerdoivek*, *kerdesek*, *eredmenyek*).

3. Komponensek és Oldalak Elemzése

3.1. Bejelentkezés Kezelése (LoginPage, AdminLoginPage)

Mindkét oldal wrapperként funkcionál a konkrét form-komponensek felett.

- **Funkció:** Sikeres bejelentkezés után (*onLoginSuccess callback*) navigálja a felhasználót a megfelelő belső oldalra (*/hianyzo-ertekelesek* vagy */admin/dashboard*).

3.2. Értékelési Folyamat (Diák oldal)

HianyzoErtekelesek

A diák itt választja ki, melyik tanárát szeretné értékelni.

- **Adatkezelés:** API hívással lekéri azon tanárok listáját, akiknél még nincs leadott értékelés.
- **UX:** Ha a lista üres, tájékoztató üzenetet jelenít meg. A gomb csak akkor aktív, ha van kiválasztott elem.

ErtekelesPage

A rendszer legösszetettebb adatbeviteli felülete.

- **Dinamikus generálás:** A backendtől kapott kérdéseket és válaszlehetőségeket mapeli ki.
- **Állapotkezelés:** A válaszok objektum kulcs-érték párokként tárolja az eredményeket (*{{kerdes_id: pont}}*).
- **Batch Submit:** A beküldéskor az objektumot tömbbé alakítja, így egyetlen kéréssel menti a teljes kérdőívet.

3.3. Adminisztrációs Modulok

AdminDashboard (Keretrendszer)

Ez a komponens felelős az admin felület reszponzív elrendezéséért.

- **Sidebar:** *NavLink*-eket használ, amelyek automatikusan megkapják az *active* osztályt.
- **Mobil-optimalizálás:** Tartalmaz egy *isOpen* állapotot, amellyel a menü kinyitható/becsukható.

AdminStats (Eredmények megtekintése)

- **Logika:** Kétlépcsős lekérdezés. Előbb a tanárlistát tölti be, majd a választott tanár ID-ja alapján lekéri a kérdésenkénti átlagokat.
- **Megjelenítés:** Strukturált táblázatban mutatja az átlagokat és a kitöltésszámot.

AdminQuestions (Kérdések szerkesztése)

Teljes CRUD felület.

- **Módosítás:** Az *isEditing* állapot váltásával ugyanazt a *formot* használja felvitelre és szerkesztésre.
- **Integritás:** A törlés előtt megerősítést kér a felhasználótól.

AdminSurveyControl (Vezérlőpult)

A kérdőív globális "ki/be" kapcsolója.

- **Működés:** A backend gyorsítótárát frissíti, ami közvetlenül befolyásolja a diákok belépési lehetőségét.

4. UI/UX és Közös Megoldások

Az összes beküldött kódban megfigyelhető egységes fejlesztői minták:

1. **Loading State Kezelés:** Minden aszinkron adatlekérés alatt a `<Loading />` komponens látható.
 - **Intelligens Pozicionálás:** A *spinner* helyzete dinamikusan változik a képernyőszélesség (*isMobile*) függvényében, elkerülve, hogy asztali nézetben a *Sidebar* alá csússzon.
 2. **Hibaellenőrzés:** A *fetch* kérések *try-catch* blokkba vannak zárva, és hiba esetén a felhasználó barátságos hibaüzenetet kap (*setError*).
 3. **Kijelentkezés (LogoutButton):** A kijelentkezés nemcsak a lokális adatokat (*localStorage*) törli, hanem megkísérli a szerveroldali token visszavonását is, ezzel zárva le biztonságosan a munkamenetet.
-